



Instituto Politécnico Nacional

Escuela Superior de Cómputo

Compiladores

Grupo:5CM2

Profra. Albortante Morato Cecilia

Equipo: Bisogno Gandarilla Ricardo

Gutiérrez Espinosa Jordan Gabriel

Ruiz Vazquez Luis Armando

Proyecto - Compilador de mi lenguaje de programación

Índice

Introducción	1
Planteamiento del problema	2
Desarrollo	2
-- Lexer.java	2
-- Sintax.java	6
-- FrmPrincipal.java	9
Resultados	13
Bibliografía	16

Introducción

Un compilador es un programa que convierte el código fuente escrito en un lenguaje de alto nivel como Java en un lenguaje legible por máquina llamado código objeto, lenguaje objeto o incluso lenguaje ensamblador. Entonces, un compilador podría llamarse traductor, pero su función es más amplia porque, como parte de la compilación de un programa, también informa errores al leer el código.

El compilador siempre revisa todo el documento como un todo. Por lo tanto, funciona de manera diferente a un intérprete. Antes de que un compilador pueda traducir el código fuente de alto nivel a un idioma de destino legible por máquina, debe escanear el código fuente y extraer los caracteres del programa. El análisis de sintaxis se realiza después del análisis léxico. Comprueba la sintaxis del código. Luego se determina la semántica del código para crear el código objeto al final. Los compiladores intentan optimizar u ocultar errores en el código. Por lo general, todas las operaciones se completan en unos pocos segundos. Es importante que el compilador continúe funcionando a pesar de los errores en el código. De lo contrario, el código fuente no se puede traducir a un idioma de destino legible por máquina.

Funciones de un compilador en programación

- **Análisis léxico:** el compilador divide el código fuente en una serie de tokens o símbolos que representan las palabras clave, identificadores, operadores y otros elementos del lenguaje de programación.
- **Análisis sintáctico:** el compilador verifica la estructura del código fuente y su conformidad con la gramática del lenguaje de programación.
- **Análisis semántico:** este verifica que las instrucciones del programa sean semánticamente correctas y coherentes, y realiza verificaciones como la asignación de tipos y la resolución de nombres.
- **Generación de código intermedio:** este representa el programa de manera más abstracta y se puede optimizar antes de generar el código objeto.
- **Optimización de código:** puede realizar una serie de optimizaciones en el código intermedio o en el código objeto para mejorar su eficiencia, como la eliminación de código redundante o la reorganización de las instrucciones para minimizar la cantidad de ciclos de procesador requeridos.
- **Generación de código objeto:** el compilador finalmente genera el código objeto que se puede ejecutar en la computadora.
- **Vinculación y carga:** si el programa utiliza bibliotecas externas, el compilador puede vincularlas al código objeto generado y generar un archivo ejecutable que se carga en la memoria de la computadora para su ejecución.

Planteamiento del problema

El reporte debe incluir:

- Las reglas de sintaxis del lenguaje de programación a compilar.
- Descripción general de cómo se realiza el análisis **léxico** en su proyecto. (Autómatas, valores de los tokens, expresiones regulares, etc)
- Descripción general de cómo se realiza el análisis **sintáctico**. (Gramáticas del lenguaje y que tipo de análisis sintáctico se aplica).
- Descripción general de cómo se realiza el análisis **semántico**.
- Pruebas de compilación con diferentes códigos del lenguaje de programación.
- Pantallas de ejecución de las pruebas de compilación, cada pantalla debe de tener su descripción de lo que se está mostrando.

Desarrollo

Lexer.java

El archivo Lexer.java es una librería diseñada para el idioma java, que se utiliza para realizar el análisis léxico de un código fuente. Al ser el que genera los "tokens", nos ayuda en el desarrollo del compilador.

El archivo Lexer.java contiene una implementación de un analizador léxico, también conocido como lexer o escáner, que lee el código fuente carácter por carácter y genera estos tokens. Cada token representa una unidad léxica, como una palabra clave, un identificador, un número, un operador, un símbolo de puntuación, etc. El lexer también puede realizar algunas tareas adicionales, como ignorar comentarios y espacios en blanco, y manejar errores léxicos cuando se encuentra una secuencia de caracteres no reconocida.

Y como se muestra en la Figura 1, se muestra como declaramos cada variable y como estos van a ser clasificados por cada token colocado en el archivo Lexer.cup que es el que contiene todas las declaraciones de tokens, algunos de los tokens que utilizamos son:

- `/* Espacios en blanco */`: Esta línea define un comentario que indica que se ignorarán los espacios en blanco en el código fuente. Los espacios en blanco incluyen espacios, tabulaciones y retornos de carro.
- `( "<<".*">>" ) {/*Ignore*/}`: Esta regla reconoce y descarta los comentarios que están entre los símbolos `<<` y `>>`. El `(.)` indica cualquier carácter y el `*` indica que puede haber cero o más ocurrencias de cualquier carácter.
- `( "\n" ) {return Linea;}`: Esta regla reconoce un salto de línea y devuelve el token "Linea".
- `( "\"" ) {lexeme=yytext(); return Comillas;}`: Esta regla reconoce una comilla doble y asigna el valor de `yytext()` a la variable `lexeme`, que representa el lexema encontrado. Luego devuelve el token "Comillas".

- (<#int | <#char | <#boolean) {lexeme=yytext(); return T_dato;}: Esta regla reconoce los tipos de datos reservados "#int", "#char" y "#boolean". Asigna el lexema encontrado a lexeme y devuelve el token "T_dato".
- ("#int") {lexeme=yytext(); return Int;}: Esta regla reconoce específicamente el tipo de dato "#int" y asigna el lexema a lexeme. Luego devuelve el token "Int".
- (<#String) {lexeme=yytext(); return Cadena;}: Esta regla reconoce el tipo de dato "#String" y asigna el lexema a lexeme. Luego devuelve el token "Cadena".
- (if) {lexeme=yytext(); return If;}: Esta regla reconoce la palabra reservada "if" y asigna el lexema a lexeme. Luego devuelve el token "If".
-
- (else) {lexeme=yytext(); return Else;}: Esta regla reconoce la palabra reservada "else" y asigna el lexema a lexeme. Luego devuelve el token "Else".
- (do) {lexeme=yytext(); return Do;}: Esta regla reconoce la palabra reservada "do" y asigna el lexema a lexeme. Luego devuelve el token "Do".
- (while) {lexeme=yytext(); return While;}: Esta regla reconoce la palabra reservada "while" y asigna el lexema a lexeme. Luego devuelve el token "While".
- (for) {lexeme=yytext(); return For;}: Esta regla reconoce la palabra reservada "for" y asigna el lexema a lexeme. Luego devuelve el token "For".
- (":") {lexeme=yytext(); return Igual;}: Esta regla reconoce el operador ":" y asigna el lexema a lexeme. Luego devuelve el token "Igual".
- ("+") {lexeme=yytext(); return Suma;}: Esta regla reconoce el operador "+" y asigna el lexema a lexeme. Luego devuelve el token "Suma".
- ("-") {lexeme=yytext(); return Resta;}: Esta regla reconoce el operador "-" y asigna el lexema a lexeme. Luego devuelve el token "Resta".
- ("*") {lexeme=yytext(); return Multiplicacion;}: Esta regla reconoce el operador "*" y asigna el lexema a lexeme. Luego devuelve el token "Multiplicacion".
- ("/") {lexeme=yytext(); return Division;}: Esta regla reconoce el operador "/" y asigna el lexema a lexeme. Luego devuelve el token "Division".
- ("&&" | "||" | "!") {lexeme=yytext(); return Op_logico;}: Esta regla reconoce los operadores lógicos "&&", "||" y "!". Asigna el lexema encontrado a lexeme y devuelve el token "Op_logico".
- (">" | "<" | "::" | "!=" | ">:" | "<:" | ">" | "<") {lexeme = yytext(); return Op_relacional;}: Esta regla reconoce varios operadores relacionales, como ">", "<", "::", "!", ">", "<", etc. Asigna el lexema a lexeme y devuelve el token "Op_relacional".
- ("+:" | "-:" | "*:" | "/:" | "%:") {lexeme = yytext(); return Op_atribucion;}: Esta regla reconoce los operadores de atribución "+:", "-:", "*:", "/" y "%:". Asigna el lexema a lexeme y devuelve el token "Op_atribucion".
- ("++" | "--") {lexeme = yytext(); return Op_incremento;}: Esta regla reconoce los operadores de incremento y decremento "++" y "--". Asigna el lexema a lexeme y devuelve el token "Op_incremento".
- (true | false) {lexeme = yytext(); return Op_booleano;}: Esta regla reconoce los literales booleanos "true" y "false". Asigna el lexema a lexeme y devuelve el token "Op_booleano".

- ("(") {lexeme=yytext(); return Parentesis_a;}: Esta regla reconoce el paréntesis de apertura "(" y asigna el lexema a lexeme. Luego devuelve el token "Parentesis_a".
- (")") {lexeme=yytext(); return Parentesis_c;}: Esta regla reconoce el paréntesis de cierre ")" y asigna el lexema a lexeme. Luego devuelve el token "Parentesis_c".

cada token será clasificado, o ignorado como los espacios y los comentarios, así tenemos toda la tablas de tokens para el analizador semántico

```
package codigo;
import static codigo.Tokens.*;
%%
%class Lexer
%type Tokens
L=[a-zA-Z_]+
D=[0-9]+
espacio=[ ,\t,\r]+
%{
    public String lexeme;
%}
%%

/* Espacios en blanco */
{espacio} { /*Ignore*/}

/* Comentarios */
( "<<"(.)*">>" ) { /*Ignore*/}

/* Salto de linea */
( "\n" ) {return Linea;}

/* Comillas */
( "\"" ) {lexeme=yytext(); return Comillas;}

/* Tipos de datos */
( "<#int | <#char | <#boolean ) {lexeme=yytext(); return T_dato;}

/* Tipo de dato Int (Para el main) */
( "#int" ) {lexeme=yytext(); return Int;}
```

```

/* Tipo de dato String */
( <#String ) {lexeme=yytext(); return Cadena;}

/* Palabra reservada If */
( if ) {lexeme=yytext(); return If;}

/* Palabra reservada Else */
( else ) {lexeme=yytext(); return Else;}

/* Palabra reservada Do */
( do ) {lexeme=yytext(); return Do;}

/* Palabra reservada While */
( while ) {lexeme=yytext(); return While;}

/* Palabra reservada For */
( for ) {lexeme=yytext(); return For;}

/* Operador Igual */
( "=" ) {lexeme=yytext(); return Igual;}

/* Operador Suma */
( "+" ) {lexeme=yytext(); return Suma;}

/* Operador Resta */
( "-" ) {lexeme=yytext(); return Resta;}

/* Operador Multiplicacion */
( "*" ) {lexeme=yytext(); return Multiplicacion;}

/* Operador Division */
( "/" ) {lexeme=yytext(); return Division;}

/* Operadores logicos */
( "&&" | "||" | "!" ) {lexeme=yytext(); return Op_logico;}

/*Operadores Relacionales */
( ">" | "<" | "==" | "!=" | ">:" | "<:" | ">" | "<" ) {lexeme = yytext(); return

/* Operadores Atribucion */
( "+:" | "-:" | "*:" | "/:" | "%:" ) {lexeme = yytext(); return Op_atribucion;}

/* Operadores Incremento y decremento */
( "++" | "--" ) {lexeme = yytext(); return Op_incremento;}

/*Operadores Booleanos*/
(true | false) {lexeme = yytext(); return Op_booleano;}

/* Parentesis de apertura */
( "(" ) {lexeme=yytext(); return Parentesis_a;}

/* Parentesis de cierre */
( ")" ) {lexeme=yytext(); return Parentesis_c;}

/* Llave de apertura */
( "{" ) {lexeme=yytext(); return Llave_a;}

/* Llave de cierre */
( "}" ) {lexeme=yytext(); return Llave_c;}

```

```

/* Corchete de apertura */
( "[" ) {lexeme = yytext(); return Corchete_a;}

/* Corchete de cierre */
( "]" ) {lexeme = yytext(); return Corchete_c;}

/* Marcador de inicio de algoritmo */
( "main" ) {lexeme=yytext(); return Main;}

/* Punto y coma */
( ";" ) {lexeme=yytext(); return P_coma;}

/* Identificador */
{L} ({L}|{D})* {lexeme=yytext(); return Identificador;}

/* Numero */
( "-" {D} "+" ) | {D}+ {lexeme=yytext(); return Numero;}

/* Funcion */
( "#" ) ({L}|{D})* {lexeme=yytext(); return Funcion;}

/* Error de analisis */
. {return ERROR;}

```

Figura 1, Analizador Léxico

Sintax.java

El archivo Sintax.java contiene la definición de la sintaxis de un lenguaje de programación en forma de reglas gramaticales utilizando la herramienta Java Cup, Siendo Javacup una herramienta que permite generar analizadores sintácticos (parsers) a partir de especificaciones en forma de gramáticas formales.

Aquí tienes un resumen de la sintaxis del programa como se muestra en la Figura 2:

El código comienza con una sección parser code que define una clase llamada Sintax que actúa como el analizador sintáctico generado.

Dentro de la clase Sintax, hay métodos y atributos definidos, como el método syntax_error que se invoca cuando ocurre un error de sintaxis y el método getS que devuelve el símbolo de entrada actual.

A continuación, se definen los terminales y no terminales utilizados en la gramática. Los terminales son tokens léxicos, como Comillas, T_dato, Int, Identificador, etc., mientras que los no terminales representan estructuras más complejas de la gramática, como INICIO, SENTENCIA, DECLARACION, etc.

La regla inicial de la gramática se especifica con start with INICIO, lo que indica que el análisis sintáctico debe comenzar desde el no terminal INICIO.

A lo largo del archivo, se definen las reglas gramaticales que describen la sintaxis del lenguaje. Estas reglas están escritas utilizando la notación de la gramática formal y especifican cómo se pueden combinar los terminales y no terminales para formar estructuras válidas del lenguaje.

Por ejemplo, se definen reglas para la declaración de variables (DECLARACION), asignaciones (ASIGNACION), estructuras de control como IF, WHILE, DO_WHILE y FOR, funciones (FUNCION), operaciones aritméticas (OPERACION), y otras construcciones del lenguaje.

Cada regla incluye un lado izquierdo (no terminal) y un lado derecho que describe cómo se pueden combinar los elementos terminales y no terminales para formar esa estructura sintáctica específica.

Las reglas también pueden contener operadores y símbolos especiales, como operadores lógicos, operadores de asignación, operadores relacionales, etc.

```
package codigo;

import java_cup.runtime.Symbol;

parser code
{
    private Symbol s;

    public void syntax_error(Symbol s){
        this.s = s;
    }

    public Symbol getS(){
        return this.s;
    }
};

terminal Comillas, T_dato, Int, Cadena, If, Else, Do, While, For,
    Igual, Suma, Resta, Multiplicacion, Division, Op_logico, Op_relacional,
    Op_atribucion, Op_incremento, Op_booleano, Parentesis_a, Parentesis_c,
    Llave_a, Llave_c, Corchete_a, Corchete_c, Main, P_coma, Identificador,
    Numero, Funcion, ERROR;
non terminal INICIO, SENTENCIA, DECLARACION, DECLARACION_FOR, IF, IF_ELSE,
    WHILE, DO_WHILE, FOR, SENTENCIA_BOOLEANA, SENTENCIA_LOGICA, SENTENCIA_LOGICA,
    SENTENCIA_FOR, FUNCION, OPERACION, PARAMETRO, ASIGNACION;

start with INICIO;

INICIO ::=
    Int Main PARAMETRO Llave_a SENTENCIA Llave_c |
    Main PARAMETRO Llave_a SENTENCIA Llave_c
;
```

```

;
PARAMETRO ::=
    Parentesis_a Identificador Parentesis_c |
    Parentesis_a Parentesis_c
;
SENTENCIA ::=
    SENTENCIA DECLARACION |
    DECLARACION |
    SENTENCIA ASIGNACION |
    ASIGNACION |
    SENTENCIA IF |
    IF |
    SENTENCIA IF_ELSE |
    IF_ELSE |
    SENTENCIA WHILE |
    WHILE |
    SENTENCIA DO_WHILE |
    DO_WHILE |
    SENTENCIA FOR |
    FOR |
    SENTENCIA FUNCION |
    FUNCION
;
ASIGNACION ::=
    Identificador Igual OPERACION P_coma |
    Identificador Igual SENTENCIA_BOOLEANA P_coma |
    Identificador Op_incremento P_coma |
    Identificador Op_atribucion OPERACION P_coma |

    Identificador Corchete_a Numero Corchete_c Op_atribucion OPERACION P_coma |
    Identificador Corchete_a Numero Corchete_c Igual OPERACION P_coma |
Identificador Corchete_a Numero Corchete_c Op_incremento P_coma |
    Identificador Corchete_a Numero Corchete_c Op_incremento P_coma |

    Identificador Corchete_a Identificador Corchete_c Op_atribucion OPERACION P_
    Identificador Corchete_a Identificador Corchete_c Igual OPERACION P_coma |
    Identificador Corchete_a Identificador Corchete_c Op_incremento P_coma |

    Identificador Op_atribucion Comillas Comillas P_coma |
    Identificador Igual Comillas Comillas P_coma |
    Identificador Op_atribucion Comillas Identificador Comillas P_coma |
    Identificador Igual Comillas Identificador Comillas P_coma
;
DECLARACION ::=
    T_dato Identificador P_coma |
    T_dato Identificador Igual OPERACION P_coma |
    T_dato Identificador Igual Op_booleano P_coma |

    T_dato Corchete_a Numero Corchete_c Identificador P_coma |
    T_dato Corchete_a Numero Corchete_c Identificador Igual OPERACION P_coma |

    Cadena Identificador Igual Comillas Comillas P_coma |
    Cadena Identificador Igual Comillas Identificador Comillas P_coma
;
OPERACION ::=
    Numero |
    Identificador |
    Numero Suma OPERACION |
    Numero Resta OPERACION |
    Numero Multiplicacion OPERACION |
    Numero Division OPERACION |
    Identificador Suma OPERACION |
    Identificador Resta OPERACION |
    Identificador Multiplicacion OPERACION |

```

```

    Parentesis_a OPERACION Parentesis_c |
    Parentesis_a OPERACION Parentesis_c Suma OPERACION |
    Parentesis_a OPERACION Parentesis_c Resta OPERACION |
    Parentesis_a OPERACION Parentesis_c Multiplicacion OPERACION |
    Parentesis_a OPERACION Parentesis_c Division OPERACION
;
IF ::= If Parentesis_a SENTENCIA_LOGICA Parentesis_c Llave_a SENTENCIA Llave_c
;
SENTENCIA_LOGICA ::=
    SENTENCIA_BOOLEANA |
    SENTENCIA_BOOLEANA Op_logico SENTENCIA_LOGICA2 |
    Identificador Op_logico SENTENCIA_LOGICA2
;
SENTENCIA_LOGICA2 ::=
    SENTENCIA_BOOLEANA |
    Identificador |
    Op_logico SENTENCIA_LOGICA2
;
SENTENCIA_BOOLEANA ::=
    Op_booleano |
    Identificador Op_relacional Op_booleano |
    Identificador Op_relacional Numero |
    Identificador Op_relacional Identificador |
    Identificador Op_relacional Comillas Comillas |
    Identificador Op_relacional Comillas Identificador Comillas
;
IF_ELSE ::= If Parentesis_a SENTENCIA_LOGICA Parentesis_c Llave_a SENTENCIA Llave_c
;
WHILE ::= While Parentesis_a SENTENCIA_LOGICA Parentesis_c Llave_a SENTENCIA Llave_c
;
DO_WHILE ::= Do Llave_a SENTENCIA Llave_c While Parentesis_a SENTENCIA_LOGICA Pare
;
FOR ::= For Parentesis_a SENTENCIA_FOR Parentesis_c Llave_a SENTENCIA Llave_c
;
SENTENCIA_FOR ::=
    T_dato Identificador Igual Numero P_coma SENTENCIA_BOOLEANA P_coma DECLARACION_FOR
    Identificador Igual Numero P_coma SENTENCIA_BOOLEANA P_coma DECLARACION_FOR
;
DECLARACION_FOR ::=
    Identificador Op_atribucion Numero |
    Identificador Op_incremento
;
FUNCION ::=
    Funcion PARAMETRO P_coma
;

```

Figura 2, Reglas de Sintaxis.

FrmPrincipal.java

es la implementación final del analizador léxico y sintáctico que creamos en Java utilizando la biblioteca JavaCC. El objetivo del .java es analizar y resaltar la sintaxis de un código fuente ingresado por el usuario y mostrar si es aceptada o no siguiendo los tokens y las reglas de sintaxis que creamos.

El código se divide en dos partes principales: la interfaz de usuario (GUI) y la lógica del analizador.

La GUI se crea utilizando componentes de la biblioteca Swing. Consiste en una ventana principal con dos secciones: "Analizador Léxico" y "Analizador Sintáctico". Cada sección tiene un área de texto para ingresar el código fuente, un botón para realizar el análisis y un botón para limpiar el área de texto como se muestra en la Figura 3.

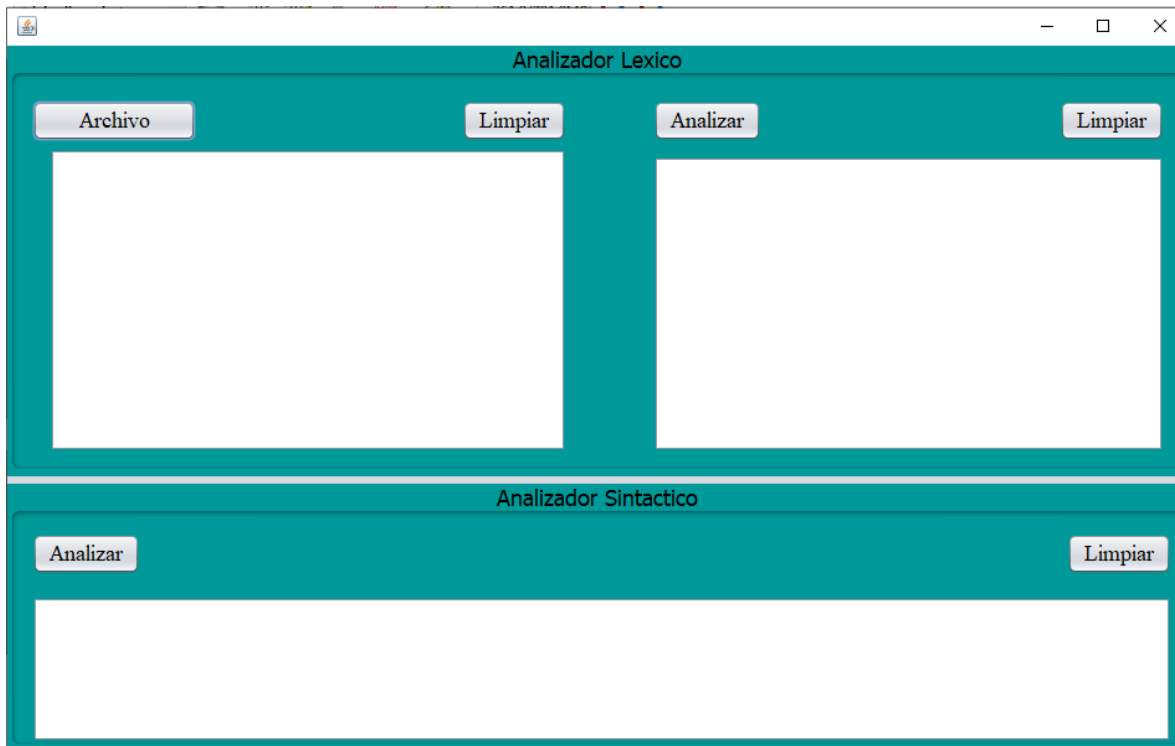


Figura 3, Interfaz gráfica.

La lógica del analizador se implementa en los métodos de la clase *FrmPrincipal*. A continuación, se explica brevemente el propósito de cada método:

analizarLexico(): Este método se utiliza para realizar el análisis léxico del código fuente ingresado. Utiliza la clase *Lexer* para analizar el código y devuelve los tokens encontrados en el área de texto de "Analizador Léxico". Los tokens reconocidos se muestran en el formato "LINEA x SÍMBOLO" seguido de una descripción donde después coloca la lista de tokens en una tabla para su análisis, Como se muestra en la Figura 4.

```

private void analizarLexico() throws IOException{
    int cont = 1;

    String expr = (String) txtResultado.getText();
    Lexer lexer = new Lexer(new StringReader(expr));
    String resultado = "LINEA " + cont + "\t\tSIMBOLO\n";
    while (true) {
        Tokens token = lexer.yylex();
        if (token == null) {
            txtAnalizarLex.setText(resultado);
            return;
        }
        switch (token) {
            case Linea:
                cont++;
                resultado += "LINEA " + cont + "\n";
                break;
            case Int:
                resultado += " <Declaracion de funcion>\t" + lexer.lexeme + "\n";
                break;
            case Comillas:
                resultado += " <Comillas>\t\t" + lexer.lexeme + "\n";
                break;
            case Cadena:
                resultado += " <Tipo de dato>\t" + lexer.lexeme + "\n";
                break;
            case T_dato:
                resultado += " <Tipo de dato>\t" + lexer.lexeme + "\n";
                break;
            case Corchete_a:
                resultado += " <Corchete de apertura>\t" + lexer.lexeme + "\n";
                break;
            case Corchete_c:
                resultado += " <Corchete de cierre>\t" + lexer.lexeme + "\n";
                break;
            case Main:
                resultado += " <Reservada main>\t" + lexer.lexeme + "\n";
                break;
            case P_coma:
                resultado += " <Punto y coma>\t" + lexer.lexeme + "\n";
                break;
            case Identificador:
                resultado += " <Identificador>\t\t" + lexer.lexeme + "\n";
                break;
            case Numero:
                resultado += " <Numero>\t\t" + lexer.lexeme + "\n";
                break;
            case Funcion:
                resultado += " <Funcion>\t\t" + lexer.lexeme + "\n";
                break;
            case ERROR:
                resultado += " <Simbolo no definido>\n";
                break;
            default:
                resultado += " < " + lexer.lexeme + " >\n";
                break;
        }
    }
}

```

Figura 4, tabla de tokens.

btnArchivoActionPerformed(): Este método se ejecuta cuando se hace clic en el botón "Archivo". Abre un cuadro de diálogo de selección de archivo para que el usuario pueda cargar un archivo de código fuente.

btnAnalizarSinActionPerformed(): Este método se ejecuta cuando se hace clic en el botón "Analizar" en la sección "Analizador Sintáctico". Utiliza la clase *Sintax* para realizar el análisis sintáctico del código fuente ingresado y muestra el resultado en el área de texto correspondiente. Si se encuentra un error de sintaxis, se muestra un mensaje de error con la ubicación y el texto del error, como se muestra en la Figura 6.

```
private void btnAnalizarSinActionPerformed(java.awt.event.ActionEvent evt) {  
    // TODO add your handling code here:  
    String ST = txtResultado.getText();  
    Sintax s = new Sintax(new codigo.LexerCup(new StringReader(ST)));  
  
    try {  
        s.parse();  
        txtAnalizarSin.setText("Análisis realizado correctamente");  
        txtAnalizarSin.setForeground(new Color(25, 111, 61));  
    } catch (Exception ex) {  
        Symbol sym = s.getS();  
        txtAnalizarSin.setText("Error de sintaxis. Línea: " + (sym.right + 1) + " Columna: " + (sym.left + 1) + ", Texto: \"" +  
        txtAnalizarSin.setForeground(Color.red);  
    }  
}
```

Figura 5, analizador sintáctico del código.

btnLimpiarSinActionPerformed(): Este método se ejecuta cuando se hace clic en el botón "Limpiar" en la sección "Analizador Sintáctico". Limpia el contenido del área de texto del analizador sintáctico.

Resultados

Utilizando un código de prueba como se muestra en la Figura 6 se muestra como quedaría nuestro código bien estructurado para que lo reconozca el compilador.

```
#int main ( ) {
    <<Comentario>>
    <#int hola : 30 + 10;
    <#int num1 : 3;
    <#boolean var: true;
    <#String nombre: "Ricardo";
    if ( nombre :: "Ricardo" ){
        while (var && nombre :: "Ricardo" ){
            num1 += 2;
            hola /= 2;
            var : false;
        }
    }
    else {
        <#int resultado : 5 + ( 6 * 8 - ( 134 / 76 ) * ( 5 + 8 ) ) ;
        #mostrar ( );
        #funcion ( resultado );
    }
    <#char[50] corrimiento;
    for(<#int recorrer :0 ; recorrer < 30 ; recorrer ++ ){
        corrimiento[recorrer] : 80 + recorrer;
        if( recorrer >: 15 ){
            do{
                <#int retardo: 0;
                retardo++;
            }while( retardo < 20);
        }
    }
}
```

Figura 6, Código para el compilador.

ya en la Figura 7 se muestra el funcionamiento del analizador Lexico y como este ordena todos los tokens que reconoce en el código.

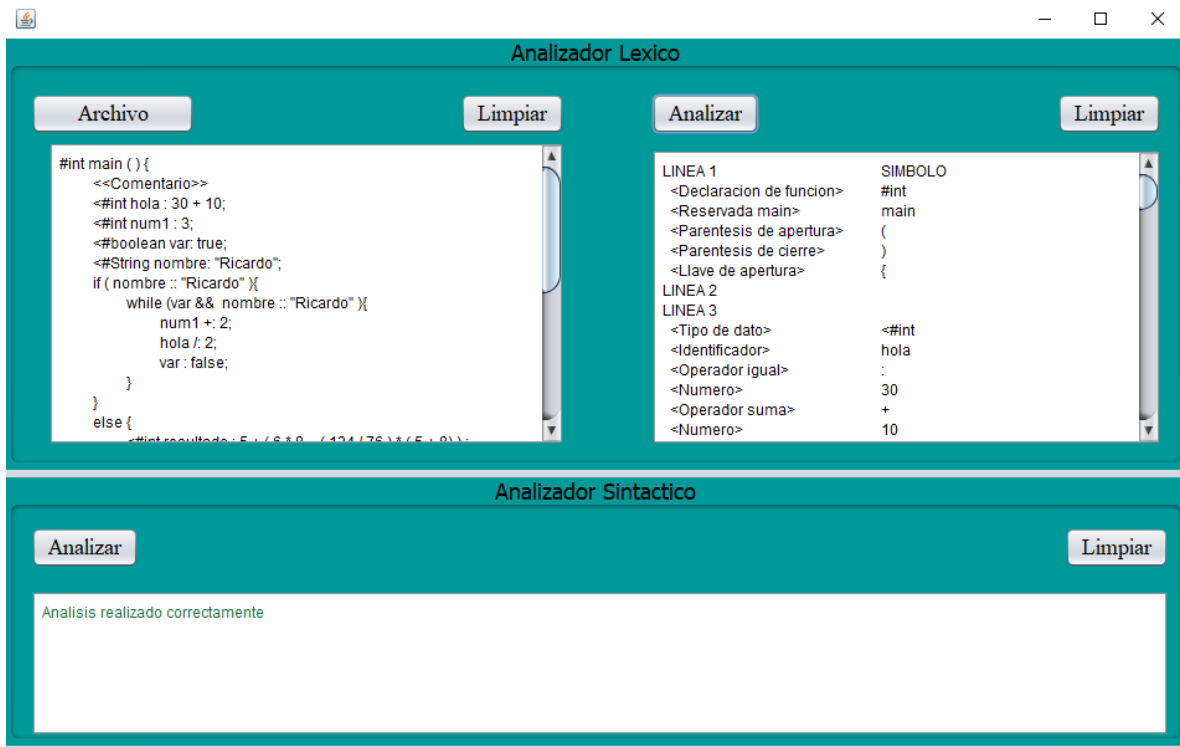


Figura 7, Funcionamiento del analizador Léxico.

Ya por último se muestra en la Figura 8 el análisis del sintáctico y como este reconoce el lenguaje que nosotros implementamos en el programa, ya por último en la Figura 9 se muestra como se hace un error en específico y el sistema lo reconoce y lo muestra en donde se encuentra.

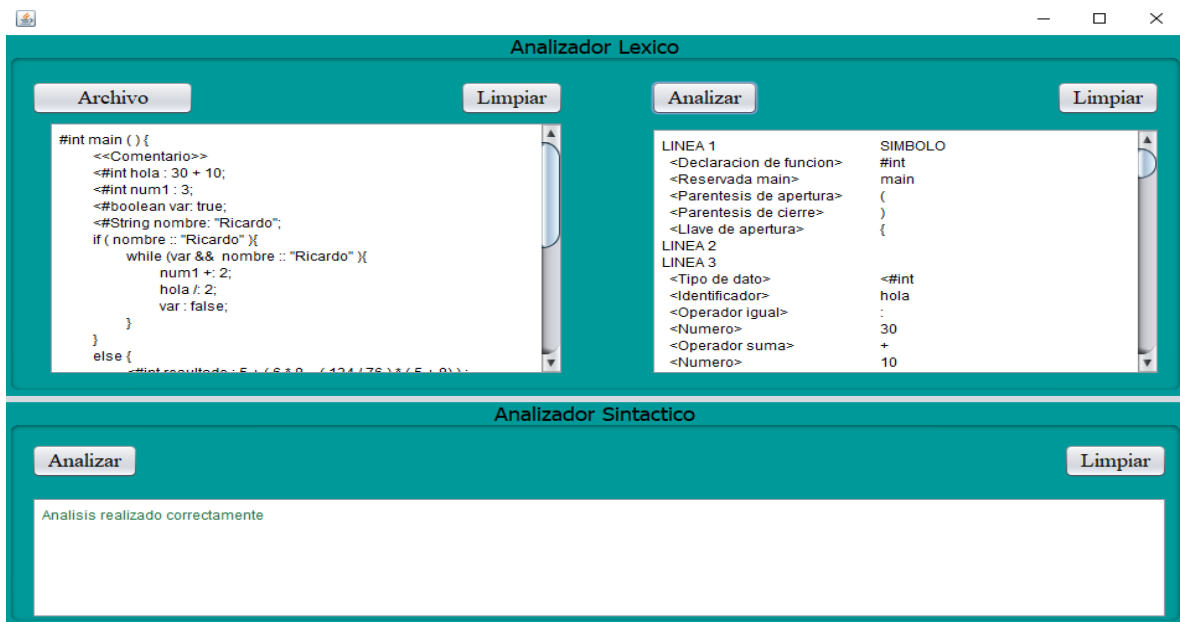


Figura 8, Funcionamiento del programa.

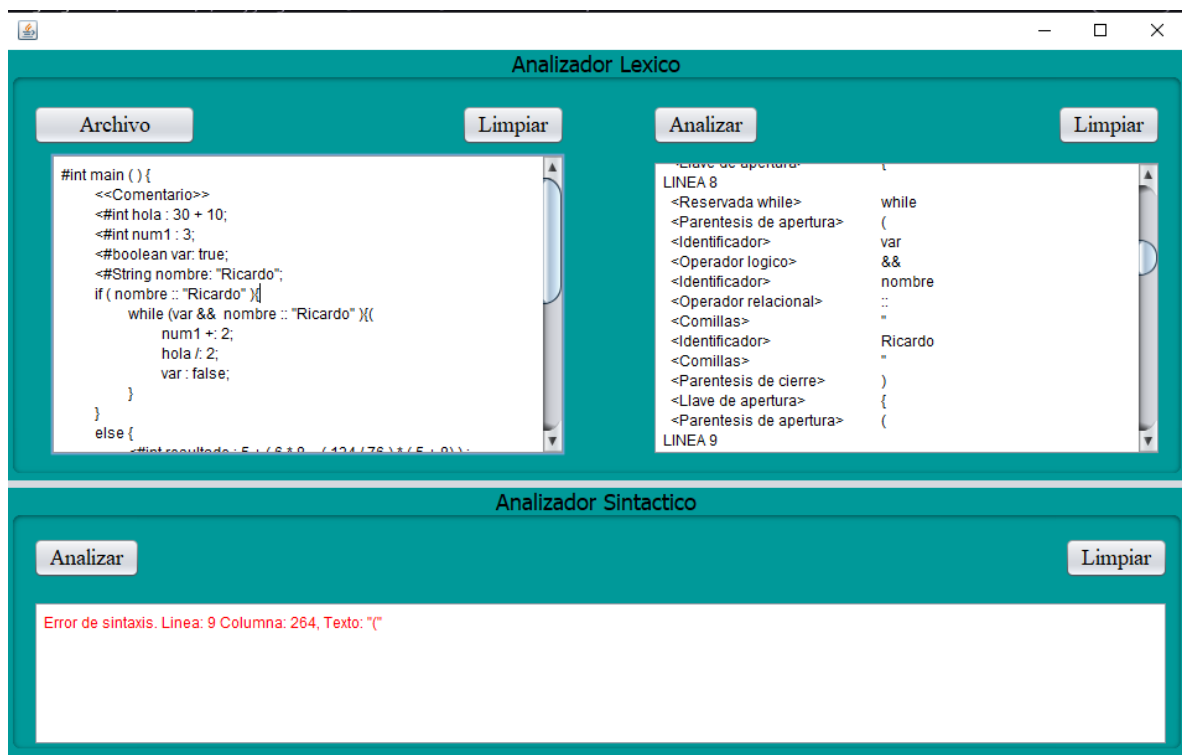


Figura 9, Error en el código y muestra de este.

Bibliografía

1.- *Compilador*. (s/f). RYTE.com. Recuperado el 26 de junio de 2023, de <https://es.ryte.com/wiki/Compilador>

2.- *Qué es un Compilador: Principales Funciones - IMMUNE*. (s/f). Immune Technology Institute. Recuperado el 26 de junio de 2023, de <https://immune.institute/blog/que-es-un-compilador/>